

Object Oriented Programming (OOP)

Semester: 2nd

Credit Hour: (01)

Department of Computer Science

Course Instructor: Engr. Zakria Bacha

Lab Task No. 7
Class & Object in C++

Previous Lab:

In previous lab, we have covered below three topics:

1. What is Class
2. C++ Constructors
3. Constructor with Parameters
4. Constructor Defined Outside the Class
5. Methods in Class
6. Constructor Overloading
7. Constructor Chaining
8. Types of Constructors in C++

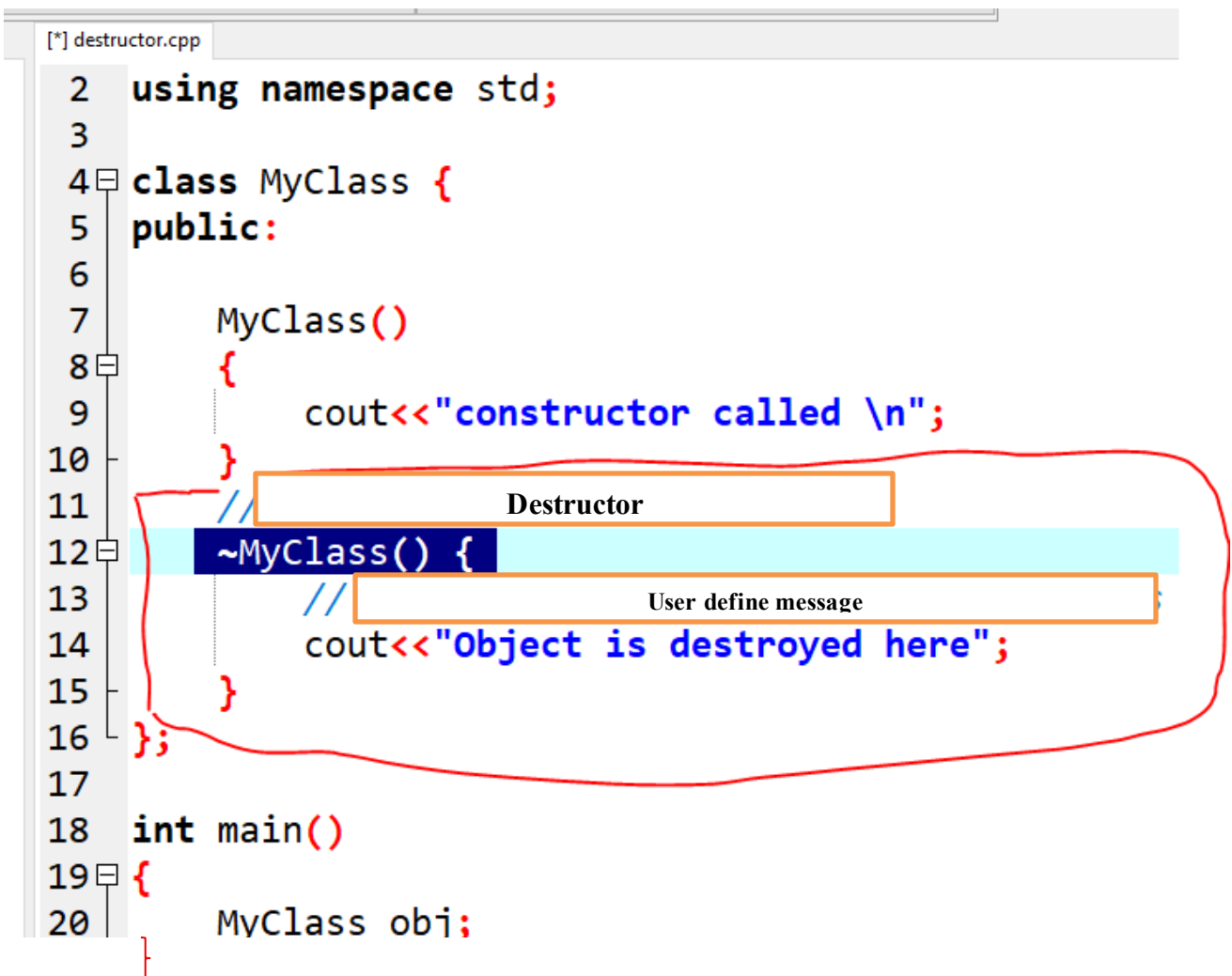
Current Topics for Lab:

1. Destructor in C++
2. Execution Order of Destructor
3. Static Data Member in Class
4. Static Methods in Class
5. Dynamic memory management

Destructors

A destructor is a special member function of a class that is executed automatically when an object of that class goes out of scope or is explicitly deleted.

Create a Destructor: To create a destructor, start by typing the tilde symbol (~) followed immediately by the exact name of the class inside the public section. You must include a pair of empty parentheses () because destructors cannot accept any arguments or parameters.



```
[*] destructor.cpp
2  using namespace std;
3
4  class MyClass {
5  public:
6
7      MyClass()
8      {
9          cout<<"constructor called \n";
10     }
11     // Destructor
12     ~MyClass() {
13         // User define message
14         cout<<"Object is destroyed here";
15     }
16 };
17
18 int main()
19 {
20     MyClass obj;
```

Figure 1

Destructor Rules

1. Use only **one** destructor inside a class. Unlike constructors, you cannot "overload" them with different versions.
2. It has **no return type** (not even **void**)
3. It is usually declared **public**.
4. We should not call it like a regular function **It is called automatically when**

- When a **local object** goes out of scope
- When a **dynamically created object** is deleted using the delete keyword

The Order of Destruction:

Destructors execute in the exact opposite order of constructors. If we create Object Student 1 then Object Student 2, and then Student 3, the program will destroy Object Student_3 first and then Object Student_2 and at last Student_3.

```
26 class Student {
27     char char_of_name;
28 public:
29     | Student(char name) : char_of_name(name) {
30         |     cout<<"Constructor " << char_of_name<<endl;
31         |     }
32     | ~Student() {
33         |     cout<<"Destructor " << char_of_name<<endl;
34         |     }
35     | };
36 int main() {
37     Student Student_1('F');
38     Student Student_2('S');
39     Student Student_3('T');
40
41     cout<<"--- End of Main function ---"<<endl;
```

Static Members:

In Object-Oriented Programming, **static** members allow you to break away from the "every object gets its own copy" rule. They are used for information or actions that belong to the **class itself** rather than any specific object.

Static Data Member

A static data member is a variable that is shared by all objects of a class. Instead of each object having its own personal variable, there is only one single copy of this variable in memory, regardless of how many objects we want to create.

Description:

1. To create a static data member, we place the keyword **static** before the variable declaration inside our class. Because this variable belongs to the class and not an instance, we must also initialize it once outside the class using the scope resolution operator **::**.
2. Once defined, every object of that class can access and modify this same variable, making it suitable for tracking shared data like a total count.

This code shows how a static variable is declared and initialized. The initialization happens outside the class.

```
48 class Student {
49
50 public:
51     static int totalStudents; // Declaration only
52     string name;
53
54     Student(string n) {
55         name = n;
56         totalStudents++; // Every time a student is created,
57                          // the SHARED count goes up
58     }
59 };
60
61 // INITIALIZATION: This is required for static members
62 int Student::totalStudents = 0;
63
```

National University of Computer and Emerging Sciences

Peshawar Campus

Static Member Function

A static member function is a specialized function used to manage static data. It can be called using the class name directly, **without needing to create an object first.**

Explanation:

1. A static member function by prefixing the function's return type with the static keyword inside the class body. These functions are unique because they do not have **this pointer**, meaning they can only interact with other static variables or static functions.
2. To call one from your main program, we should use the class name followed by two colons and the function name, such as `ClassName :: FunctionName()`.

```
73 class School {
74 public:
75     static string schoolName;
76
77     // Static Function: Can be called using School::getSchoolName()
78     static void displaySchool() {
79         cout << "Welcome to: " << schoolName << endl;
80     }
81 };
82
83 string School::schoolName = "FAST National University";
84 int main()
85 {
86     School::displaySchool();
87 }
```

Access:

Static functions **cannot** access regular variables (like name or age) because they don't know which object those variables belong to.

Memory:

Static members are stored in a separate static memory area, not inside the memory block of the objects.

Global vs Static:

A static member is like a global variable, but it is "hidden" inside a class to keep the code organized and secure.

National University of Computer and Emerging Sciences
Peshawar Campus

Correct or not?

```
73 class School {  
74 public:  
75     static string schoolName;  
76     int no_student;  
77  
78     // Static Function: Can be called using School::getSchoolName()  
79     static void displaySchool() {  
80         cout << "Welcome to: " << schoolName << endl;  
81         cout << no_student;  
82     }  
83 };  
84  
85 string School::schoolName = "FAST National University";  
86 int main()  
87 {  
88     School::displaySchool();
```

Correct or not?

```
73 class School {  
74 public:  
75     static string schoolName;  
76  
77     // Static Function: Can be called using School::getSchoolName()  
78     static void displaySchool() {  
79         schoolName = "FAST";  
80         cout << "Welcome to: " << schoolName << endl;  
81     }  
82 };  
83  
84 //string School::schoolName = "FAST National University";  
85 int main()  
86 {  
87     School::displaySchool();  
88 }
```

Complete Student Class Program:

It combines **Constructors**, **Destructors**, **Static Data**, and **Static Functions** into one logical flow.

```
97 class Student {  
98 private:  
99     string name;  
100     int id;  
101     static int studentCount; // Static Data Member  
102 public:  
103     // Constructor  
104     Student(string n, int i) {  
105         name = n;  
106         id = i;  
107         studentCount++; // Increment shared counter  
108         cout<<"Student"<<name<<"enrolled."<<endl;  
109     }  
110
```

Figure 2

```
1 // Destructor  
2 ~Student() {  
3     studentCount--; // Decrement shared counter when object  
4     cout<<"Student " <<name<<"record removed."<<endl;  
5 }  
6  
7 // Static Member Function  
8 static int getCount() {  
9     ✓ return studentCount; // Returns the shared variable  
0 }
```

Constructor/Destructor: They act as the "Entry" and "Exit" points that automatically manage the counter.

Static Function: getCount() allows the user to check the status without needing to "ask" a specific student object

```
122 // Initialize static member
123 int Student::studentCount = 0;
124 int main() {
125
126     cout<<"Initial Count\n"<<Student::getCount()<<"\n";
127
128     return 0;
129 }
```

Output?

```
128 main()
129 // Creating students in a block scope
130 {
131     Student s1("Student 1", 101);
132     Student s2("Student 2", 102);
133     cout << "Current Count: " << Student::getCount()<<"\n";
134 } // s1 and s2 go out of scope and their destructors are called
135
```

Output?

National University of Computer and Emerging Sciences
Peshawar Campus

Output?

```
124 int main() {  
125  
126     cout<<"Initial Count\n"<<Student::getCount()<<"\n";  
127  
128     main()  
129     // Creating students in a block scope  
130     {  
131         Student s1("Student 1", 101);  
132         Student s2("Student 2", 102);  
133         cout << "Current Count: " << Student::getCount()<<"\n";  
134     } // s1 and s2 go out of scope and their destructors are called  
135  
136     cout<<"Final Count after graduation\n"<<Student::getCount()<<"\n";  
137  
138     return 0;  
139 }
```

Combined Output:

```
C:\Users\Admin\Desktop\Constructor_Practice\destructor.exe  
Initial value of Count  
0  
  
Student Student 1      enrolled.  
Student Student 2      enrolled.  
  
Current Count  2  
  
Student      Student 2      record removed.  
Student      Student 1      record removed.  
  
Final Count after graduation  
0  
-----  
Process exited after 0.2854 seconds with return value 0  
Press any key to continue . . .
```

Correct or wrong?

```
4 class Student {
5 private:
6     int* data; // A pointer to hold an address on the heap
7
8 public:
9     // Constructor Allocates memory manually using 'new'
10    MemoryManager() {
11        data = new int; // Requesting memory from the heap
12        *data = 100;    // Assigning a value to that memory
13        cout<<"Constructor Memory allocated on the heap.";
14    }
15
16    // Destructor
17    ~MemoryManager() {
18
19        cout<<"Destructor Memory freed." << endl;
20    }
```

1. Destruction of Object and Pointer Variable

When an object is destroyed, all its member variables (including pointer variables) are automatically removed. The pointer itself is deleted from memory because it is part of the object.

2. Manual Deallocation of Dynamic (Heap) Memory

Memory allocated using new exists in the heap and is not automatically removed. Therefore, we must manually free this memory using delete, usually inside the destructor.

3. Destructor Does Not Automatically Free Heap Memory

A destructor does not automatically release dynamically allocated memory. If delete is not used inside the destructor, the heap memory remains allocated, causing a memory leak.

```
// Deallocates memory manually using 'delete'
~MemoryManager() {
    delete data; // Releasing the heap memory back to the OS
    cout<<"Destructor Memory successfully freed."<<endl;
}
```

Lab Task No. 1:

First, create a **BankAccount** class where all objects share a private static variable called *totalBalance* that stores the combined money of all accounts. The constructor should take an initial deposit and add it to this shared total, while the destructor should display a message like “Account Closed” when an object is destroyed. Moreover, include a public static function to show the current total balance so you can easily track how all objects contribute to one shared value.

```
=====
=====
```

Task 2:

Create a **LibraryBook** class where only **5 books** can be kept in the system at a time. You should use a private static counter variable to keep track of how many books are currently added. In the constructor, check the current count; if it is less than 5, increase the counter and create the object, otherwise show a message like “Library is full, cannot add more books.” In the destructor, decrease the counter when a book is removed or returned. Also include a public static function to show the current number of books.

```
=====
=====
```

Task 3:

Create an **Employee** class with a maximum limit of **10 employees** using a private static counter. In the constructor, create a member function to check if the count is less than 10; if true, increase the count, otherwise show “No more employees can be added.” In the destructor, decrease the count when an object is destroyed. Also include a static function to show the current number of employees.

```
=====
```

National University of Computer and Emerging Sciences

Peshawar Campus

